

PROYECTO PROGRAMACION ORIENTADA A OBJETOS



Nombre: Isaac Lopez Simental
Clase: Programacion Orientada a Objetos
Unidad: 1
Cuatrimestre: 4
Grupo 4B Bis



ÍNDICE

ÍNDICE.....	1
Descripción del programa.....	1
Idea general.....	2
Clases.....	3
Instancias.....	4
Herencia.....	5
Polimorfismo.....	6
Funcionamiento general.....	7



Descripción del programa

El programa representa un sistema básico de entidades inspirado en criaturas de videojuegos. Su función principal es permitir la creación y administración de distintos tipos de mobs dentro de una misma aplicación.

Durante la ejecución, el usuario puede crear nuevas entidades, consultar cuáles existen, revisar información específica de alguna de ellas y eliminar mobs de la lista. Además, cada tipo de entidad puede contar con comportamientos particulares, especialmente al realizar acciones como atacar.

El objetivo principal del proyecto no es solamente administrar criaturas, sino demostrar de manera práctica varios conceptos fundamentales de la Programación Orientada a Objetos. Entre ellos se encuentran el uso de clases, objetos, herencia, métodos sobrescritos y polimorfismo.

Idea general

La estructura del programa parte de una entidad general llamada **Mob**, que funciona como base para representar las características comunes de todas las criaturas.

A partir de esta estructura general se crean diferentes tipos de mobs, como **Skeleton**, **Creeper**, **Zombie** y **Drowned**. Cada uno comparte ciertas características, pero también puede tener comportamientos propios.

La clase **Main** es la encargada de controlar la interacción con el usuario. Desde ahí se muestra el menú principal, se reciben las opciones seleccionadas y se realizan las operaciones sobre las entidades almacenadas.

En conjunto, el programa permite manejar diferentes objetos de manera organizada y mantener una estructura clara entre las partes generales y las específicas del sistema.



Clases

El proyecto está dividido en varias clases, cada una con una responsabilidad diferente dentro del programa.

La clase **Main** controla el flujo general del sistema. Se encarga del menú, de recibir las opciones del usuario y de administrar las entidades que se van creando durante la ejecución.

La clase **Mob** representa la base común de las entidades. En ella se agrupan las características que todos los mobs comparten, como su identificación, nombre, vida, velocidad o estado.

Después se encuentran las clases específicas de cada tipo de criatura. Estas clases aprovechan la estructura existente y agregan o modifican comportamientos de acuerdo con el tipo de mob que representan.

Esta separación permite que el programa sea más organizado y facilita modificar o ampliar el sistema posteriormente.



Instancias

Las instancias son los objetos reales que se crean durante la ejecución del programa a partir de las clases existentes.

Por ejemplo, cuando el usuario selecciona la opción para crear un mob, el programa genera una nueva entidad de alguno de los tipos disponibles. Esa entidad pasa a ser un objeto independiente dentro del sistema.

Cada instancia conserva sus propios datos, aunque pertenezca a la misma clase que otras entidades. Esto permite que existan varios mobs del mismo tipo sin que sean exactamente el mismo objeto.

Las instancias creadas se almacenan dentro de una lista, lo que permite al programa recorrerlas, mostrarlas, identificarlas y eliminarlas cuando sea necesario.

Herencia

La herencia es uno de los conceptos principales utilizados en este proyecto.

Permite que las clases específicas aprovechen las características definidas en una clase más general. De esta forma, no es necesario repetir la misma información en cada tipo de entidad.

La estructura principal de herencia es:

Mob → Skeleton

Mob → Creeper

Mob → Zombie → Drowned

Esto significa que **Skeleton**, **Creeper** y **Zombie** reciben las características generales de **Mob**.

En el caso de **Drowned**, existe un nivel adicional de herencia, ya que hereda de **Zombie**, que a su vez ya hereda de **Mob**. Por eso se puede considerar un ejemplo de herencia multinivel.

Gracias a esta estructura, el programa reutiliza código y mantiene una relación lógica entre los diferentes tipos de criaturas.



Polimorfismo

El polimorfismo permite que diferentes tipos de entidades sean manejados de forma común dentro del programa.

Aunque cada mob pertenece a una clase diferente, todos pueden ser tratados como entidades de tipo **Mob** debido a la relación de herencia.

Esto es especialmente útil para almacenarlos dentro de una misma lista y trabajar con todos ellos utilizando una estructura general.

Al mismo tiempo, cada clase puede tener su propia forma de responder a ciertas acciones. Por ejemplo, dos mobs diferentes pueden realizar un ataque de manera distinta aunque ambos sean tratados de forma general como **Mob**.

Por lo tanto, el polimorfismo permite combinar una administración común de las entidades con comportamientos específicos para cada tipo de criatura.

Funcionamiento general

El programa inicia mostrando un menú principal al usuario. Desde este menú se pueden seleccionar diferentes acciones relacionadas con las entidades.

Cuando se crea un mob, este se agrega a la lista principal del sistema. Posteriormente, el usuario puede consultar las entidades disponibles, seleccionar una de ellas para mostrar su información o eliminarla.

El programa se mantiene en funcionamiento mediante un ciclo que vuelve a mostrar el menú después de realizar cada operación. Este proceso continúa hasta que el usuario selecciona la opción para salir.

De esta manera, el sistema combina una estructura sencilla de interacción con una organización basada en los principios de la Programación Orientada a Objetos.

